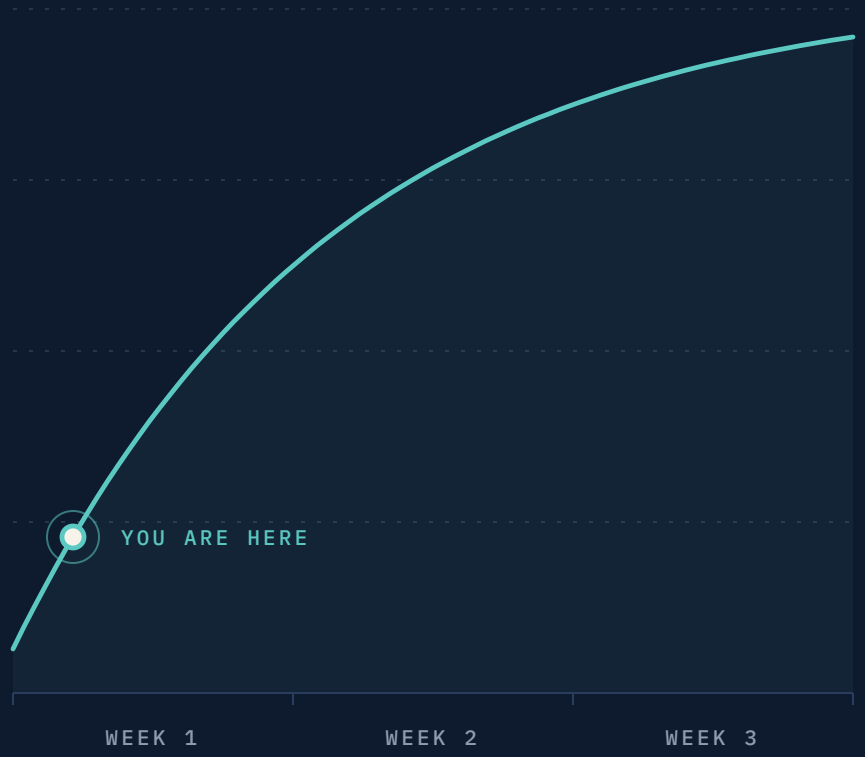
WEEK 1 · DAY 2 · OVERVIEW

JSON and the FRED API



Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 10 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

Yesterday FRED's answer came back as text, its yields in quotes. Make it a dated series the desk can trust. Show me what is in the answer, and what FRED writes on a weekday with no yield. Prove a missing day never becomes a zero: show me what a zero would do. Put the request, the reading and the download in marketdata.py, tested without the network, with errors that never show a key. Pull the 10-year and the 2-year from 2015 and tie them to the Treasury file, inside FRED's limit. Then read exactly what you are about to commit, and scan it for anything shaped like a key.

ABOUT 50 MIN WITH EXERCISES / 10 SECTIONS / 4 EXERCISES

Your Route Through Day 2

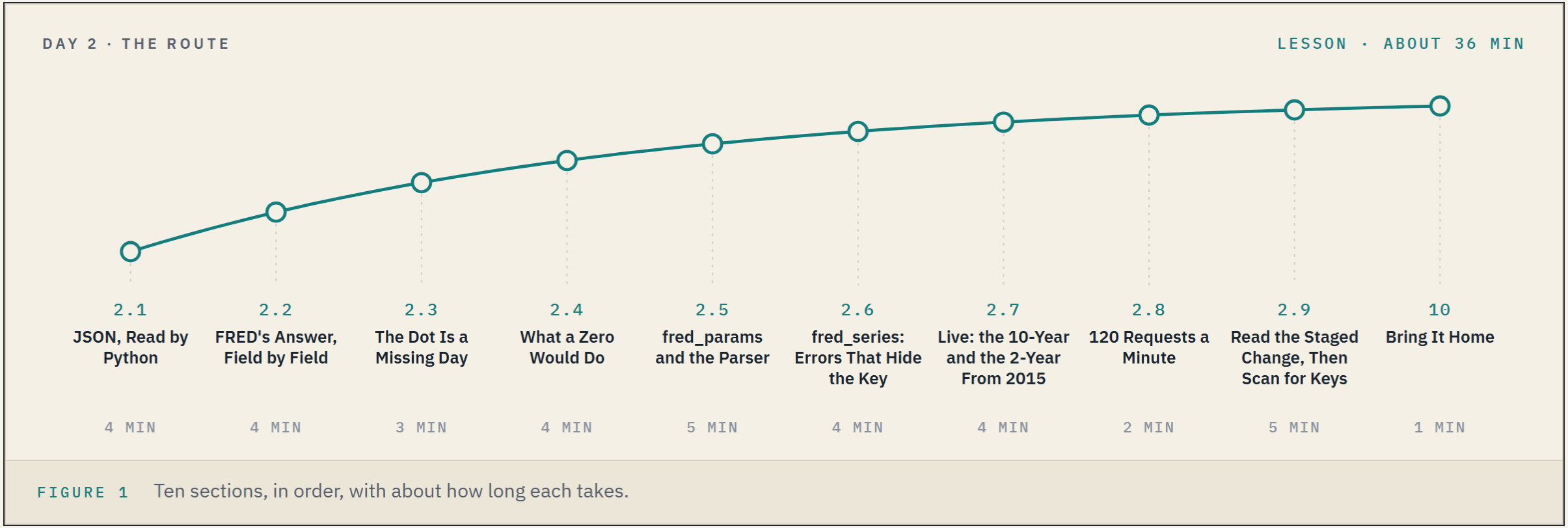


FIGURE 1 Ten sections, in order, with about how long each takes.

Time. About 36 minutes for the lesson, then about 12 for the four exercises.

2.1 JSON, Read by Python

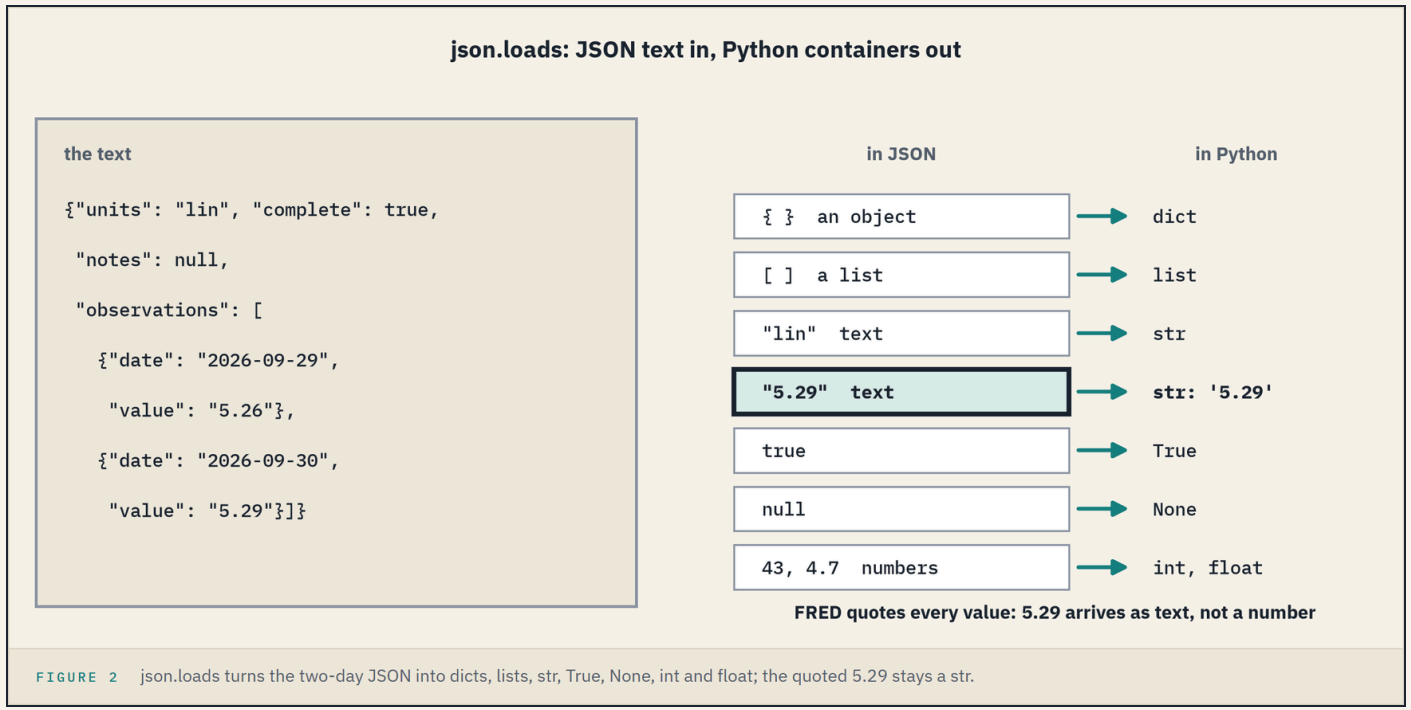


FIGURE 2 json.loads turns the two-day JSON into dicts, lists, str, True, None, int and float; the quoted 5.29 stays a str.

READ THE TEXT

```
reply = json.loads(text)
print(type(reply), type(reply['observations']))
```

```
<class 'dict'> <class 'list'>
```

WHAT EACH VALUE BECAME

```
units: 'lin', str
complete: True, bool
notes: None, NoneType
value: '5.29', str
```

JSON is two containers and a few plain values; `json.loads` makes them a `dict` and a `list`, and FRED's 5.29 arrives as the text `'5.29'`.

2.2 FRED's Answer, Field by Field

The shape of FRED's answer: 12 fields, then the observations

12 fields about the answer

```
realtime_start: "2026-10-02"
realtime_end: "2026-10-02"
observation_start: "2026-08-03"
observation_end: "2026-09-30"
units: "lin"                    levels, as published
output_type: 1
file_type: "json"
order_by: "observation_date"
sort_order: "asc"
count: 43                       observations in the list
offset: 0                       starting at the first
limit: 100000                   most per answer
```

observations: a list of 43

```
 0  date: "2026-08-03"  value: "4.70"
 1  date: "2026-08-04"  value: "4.63"
 ...
24  date: "2026-09-04"  value: "4.78"
25  date: "2026-09-07"  value: "."
26  date: "2026-09-08"  value: "4.80"
 ...
42  date: "2026-09-30"  value: "5.29"
```

each also carries realtime_start and realtime_end

FIGURE 3 FRED's answer has 12 fields and 43 observations; row 25, 2026-09-07, holds a dot instead of a yield.

READ THE SAMPLE

```
print('count:', payload['count'])
```

```
count: 43
```

EVERY COLUMN IS TEXT

```
print(obs.dtypes)
```

```
realtime_start    str
realtime_end      str
date              str
value             str
dtype: object
```

Twelve fields describe the answer; the 43 observations sit in a list of dicts.

The Dot on Labor Day

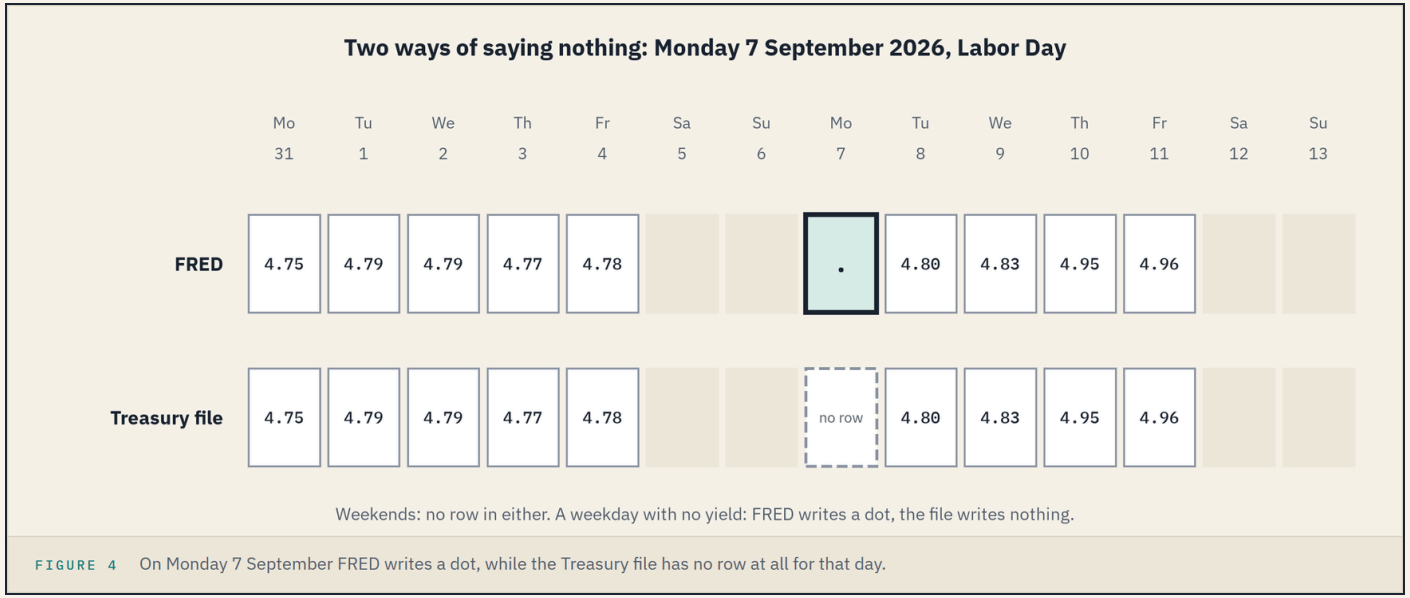


FIGURE 4 On Monday 7 September FRED writes a dot, while the Treasury file has no row at all for that day.

THE ROWS AROUND THE DOT

	DATE	VALUE
23	2026-09-03	4.77
24	2026-09-04	4.78
25	2026-09-07	.
26	2026-09-08	4.80
27	2026-09-09	4.83

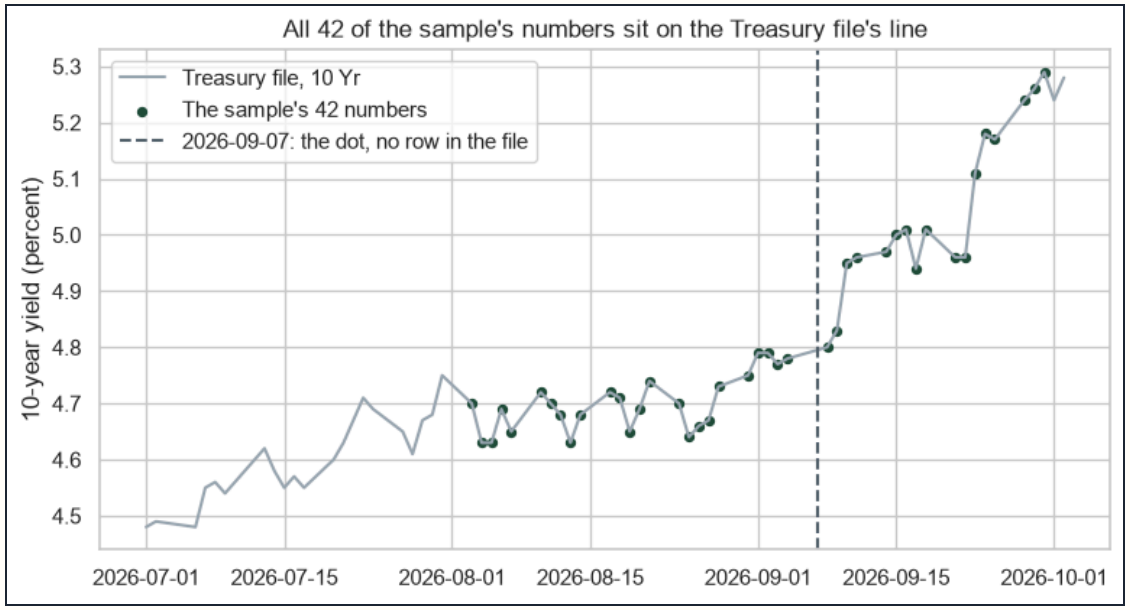
EXCEL SIDE BY SIDE

JOB	EXCEL
Count the rows	`=COUNTA(B2:B44)`
Count the dots	`=COUNTIF(B2:B44,".")`

FRED writes a row with a dot on a weekday with no yield, where the Treasury file writes no row at all: count rows and you count the dot.

2.3 The Dot Is a Missing Day

SAMPLE ON THE SNAPSHOT



READ THE DOT AS A NUMBER

```
float('.')
```

```
ValueError: could not convert string to float: '.'
```

MARK IT MISSING

```
kept = pd.to_numeric(obs['value'], errors='coerce')
```

```
float64 | 1 NaN, on 2026-09-07
```

EXCEL SIDE BY SIDE

JOB	EXCEL	RESULT
Read the dot as a number	`=VALUE(".")`	`#VALUE!`

42 numbers equal to the Treasury file, and the 43rd row is a day the market had no yield: the dot is a missing day, nothing else.

2.4 What a Zero Would Do

THE EASY FIX

The easy fix is a 0: the column stays numeric and nothing stops.

AVERAGE BOTH WAYS

```
print(f'Dot dropped: {sample_10y.mean():.4f} percent')
print(f'Dot as 0:    {as_zero.mean():.4f} percent')
print(f'Gap: {(sample_10y.mean() - as_zero.mean()) * 100:.1f} bp')
```

```
Dot dropped: 4.8367 percent
Dot as 0:    4.7242 percent
Gap: 11.2 bp
```

EXCEL SIDE BY SIDE

EXCEL	RESULT
`Table.TransformColumnTypes(Rows, {{"date", type date}, {"value", type number}}, "en-US")`	43 rows; the dot loads as an empty cell
`=AVERAGE(B2:B44)`	4.836666666666667: the text skipped
`=SUM(B2:B44)/ROWS(B2:B44)`	4.724186046511628

2.5 fred_params and parse_fred_observations

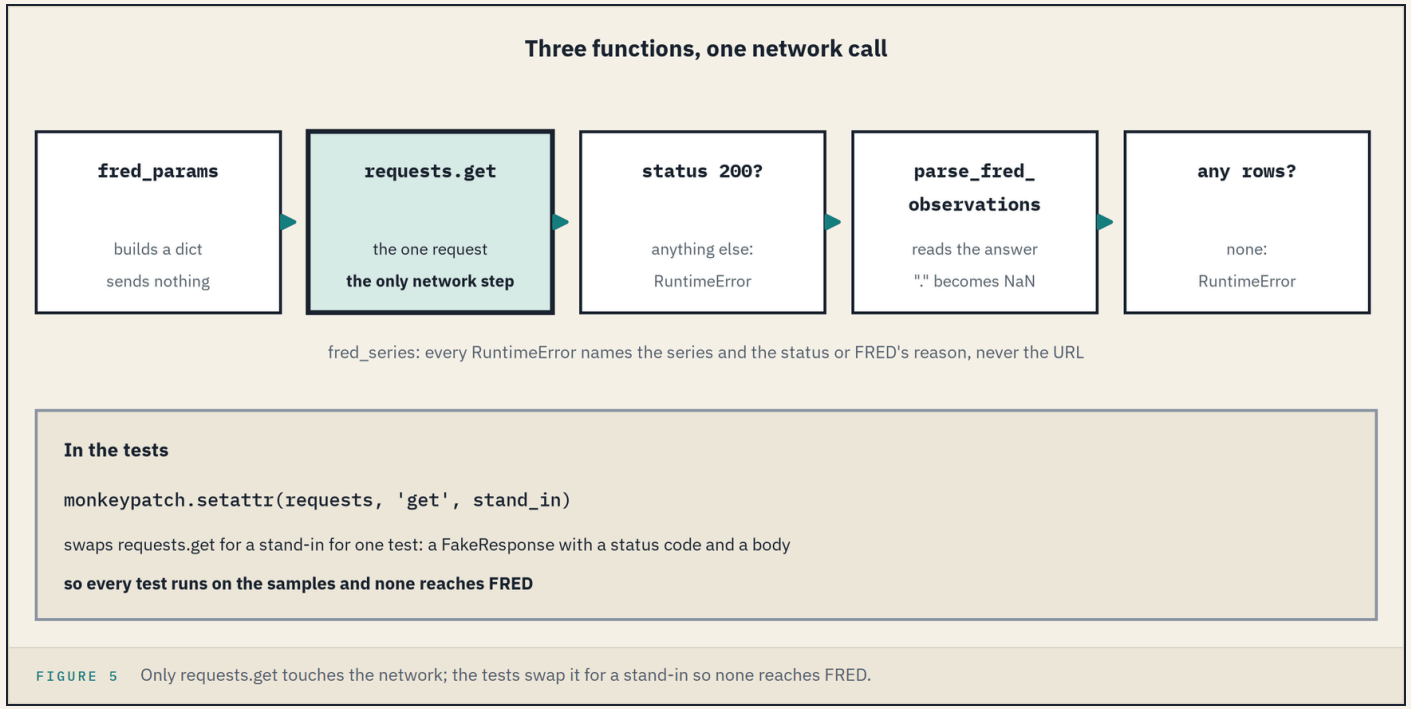


FIGURE 5 Only requests.get touches the network; the tests swap it for a stand-in so none reaches FRED.

THE PARSER CALL

```
dgs10 = md.parse_fred_observations(payload, 'DGS10')
```

WHAT IT PRINTED

```
43 rows, 1 NaN, float64 | index: date
Same numbers as 2.3 by hand: True
```

Two functions, neither touching the network: one builds the request, one reads the answer with the dot kept as `NaN`, and `7 passed` proves both.

2.6 fred_series: Errors That Hide the Key

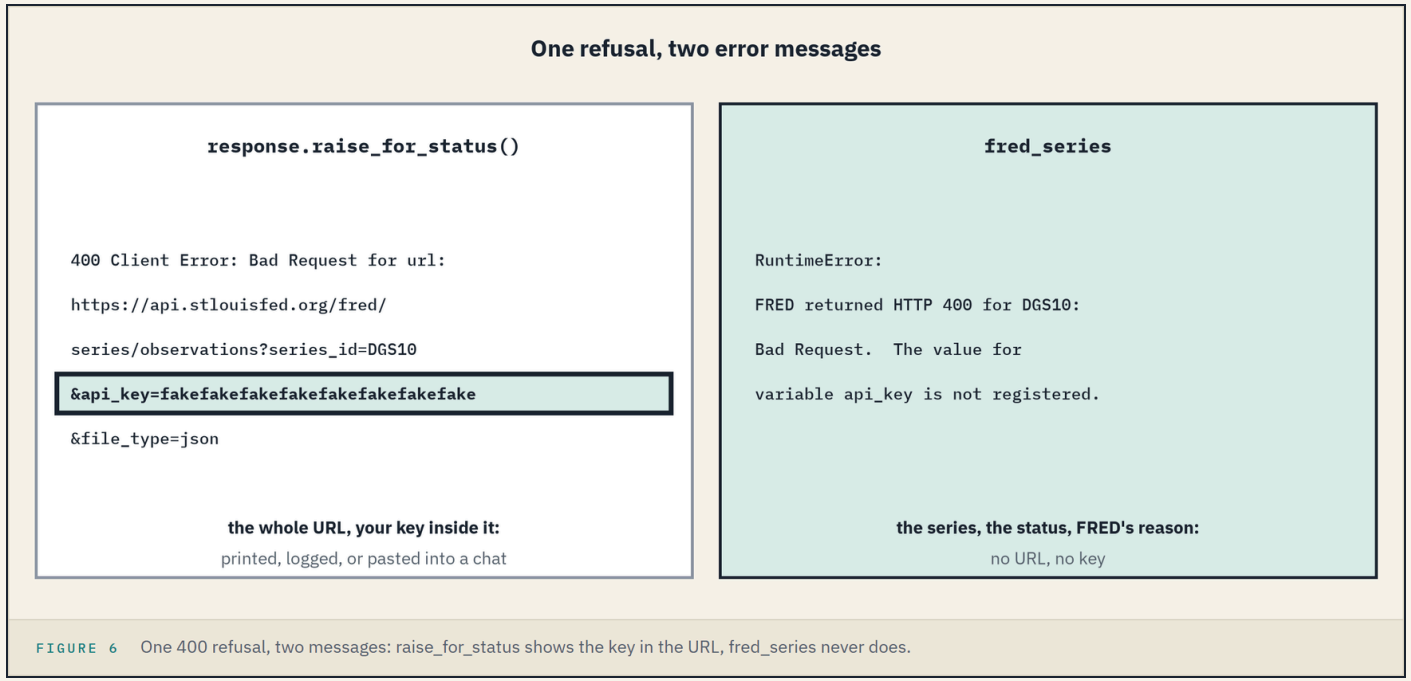


FIGURE 6 One 400 refusal, two messages: raise_for_status shows the key in the URL, fred_series never does.

SWAP THE NETWORK OUT

```
monkeypatch.setattr(requests, "get", fake_get_400)
```

IN THE MODULE

```
fred_series is in the module: True
```

`fred_series` **reports the status and FRED's own reason, never the URL, and a test that formats the whole error proves no key survives in it.**

2.7 Live: the 10-Year and the 2-Year From 2015

Same numbers, two timetables

	Mon 28 Sep	Tue 29 Sep	Wed 30 Sep	Thu 01 Oct	Fri 02 Oct
Treasury file, 10 Yr retrieved 2026-10-03	5.24	5.26	5.29	5.24	5.28
FRED, DGS10 checked 2026-10-04	5.24	5.26	5.29	5.24	not there yet

The check compares only the dates both have; the newest day can sit on one source first.

FIGURE 7 FRED's DGS10 matches the Treasury file on four days of the week of 28 September 2026.

WITH YOUR KEY

With your key, the first cell prints FRED's refusal, status and reason, and no key; the second prints one line per series: its rows since 2015, its dots, its last date, and the dates it shares with the file, at a largest gap of 0.

THE PULL

```
live_series = md.fred_series(live_id, fred_key, start='2015-01-01')
live_numbers = live_series.dropna()
```

With your key, both series pull through `fred_series` and equal the Treasury file on every date both have.

2.8 120 Requests a Minute

THE RULE

FRED: "Up to 120 requests per minute are allowed before being served a 429 error code" (https://fred.stlouisfed.org/docs/api/fred/errors.html, read 2026-11-08).

COUNT THE REQUESTS

```
runs = list(range(1, 13))
requests_made = [11 * run for run in runs]
```

DRAW THEM

```
bars = ax.bar(runs, requests_made, color=colours, tick_label=runs)
ax.bar_label(bars, padding=2)
ax.set_ylim(0, 145)
ax.set_xlabel('Runs of the 11-tenor loop within one minute')
ax.set_ylabel('Requests sent')
```

2.9 Read the Staged Change, Then Scan for Keys

SCAN FOR A KEY

```
!git -C "{work}" grep --cached -n -E "[a-z0-9]{{32}}"
```

```
test_marketdata.py:12:FAKE_KEY = "fakefakefakefakefakefakefakefake"
```

COMMIT

```
!git -C "{work}" commit -q -m "{subject}" -m "{body}"
!git -C "{work}" log --oneline
```

```
f3002d2 Notebook 02: fred_params, parse_fred_observations, and fred_series
a28686d Notebook 01: get_secret, a .gitignore, and .env.example
```

The commit holds two files, read line by line and scanned: one 32-character run, the course's fake key, where the tests keep it.

Bring It Home

IN A POWERSHELL TERMINAL

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_02_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py" .
# the samples: read by the tests, never committed
Copy-Item "$work\course3_fred_format_dgs10.json", "$work\course3_fred_format_dgs2.json" .
python -m pytest -q test_marketdata.py
git add marketdata.py test_marketdata.py
git diff --cached --stat
git grep --cached -n -E "[a-z0-9]{32}"
git commit -m "Notebook 02: fred_params, parse_fred_observations, and fred_series"
```

You Can Now

- Read JSON with **`json.loads`**, and say what each piece becomes in Python.
- Find FRED's missing marker, the dot, and say why the Treasury file has no row there.
- Keep a missing day as **`NaN`**, drop it in a step you can see, and show what a 0 would do.
- Build a request with **`fred_params`** and read an answer with **`parse_fred_observations`**, tested without the network.
- Download a series with **`fred_series`**, whose errors never show the key, proved with **`monkeypatch`**.
- Tie a live pull to the Treasury file, and stay inside FRED's 120 requests a minute.
- Read a staged change with **`git diff --cached`**, and scan it for a key with **`git grep`**.

NEXT
WEEK 1 · DAY 3

Market Calendars and Cleaning

The market's days are the dates the Treasury file has: the weekdays it skips, a holiday calendar that disagrees with it, FRED's dots on the same days, series joined without inventing a day, forward fill kept for display, and values that stopped moving.